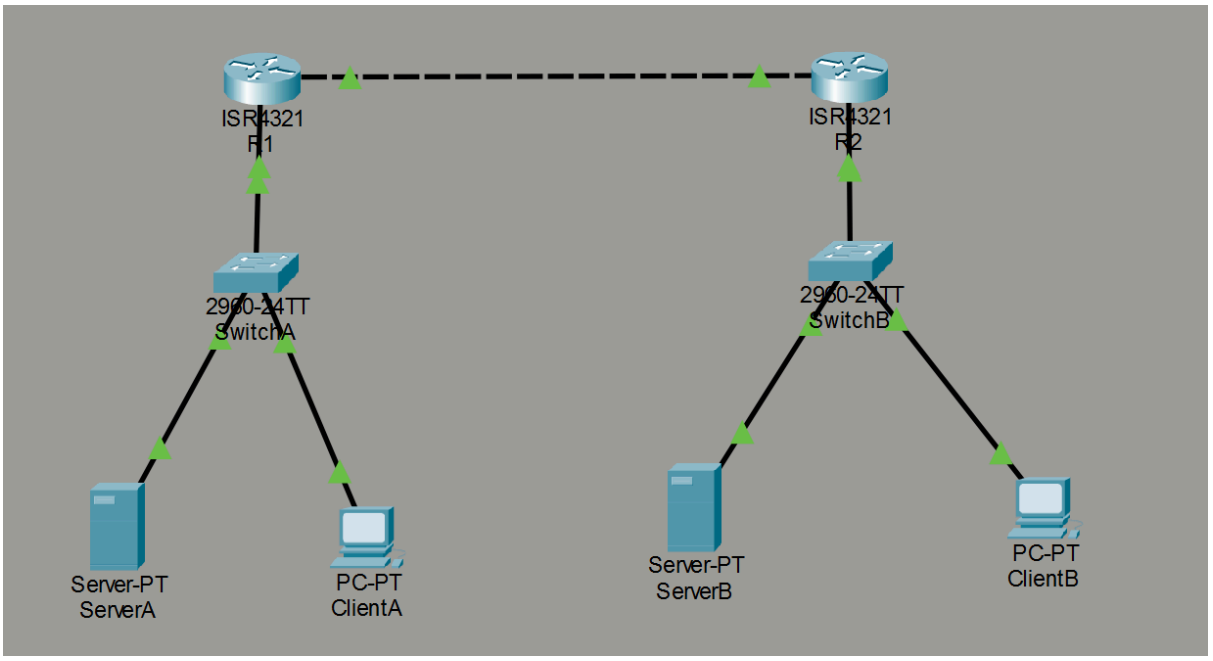
Lab 4



สมาชิกกลุ่ม

กรกฎ พรมทอง (บิ๊ก) 673380025-8
จิณณวัตร โพธิ์ศรีทอง (มาร์ช) 673380263-2
วัชรวิศว์ น้อยเมล์ (โมเดิร์น) 673380059-1
ปกรณ์เกียรติ ศรีจันทร์ (เปรม) 673380045-2
สรวิชญ์ ทะมานันท์ (โบ๊ท) 673380295-9

Lab 4 using your specific repository, layout, and networking assumptions. This version **treats Local subnets as private networks** and **treats the remote server / inter-site network as a 10.10.x.x "internet/public network"** for the purposes of NAT and routing logic. It also assumes the repository hierarchy:

```
automation/
├── mockup-infra/
└── week02-stateless-stateful/
    └── phase1-mockup/
```

We will produce:

✅ A revised **high-fidelity lab plan** ✅ **Detailed addressing & topologies** ✅ **Cisco-themed network section** ✅ **IaC / VS Code section** ✅ **Troubleshooting and engineering reflection** ✅ A complete **README.md** ready to be committed to the repo

---

🧪 LAB 4 Simulated Internet (10.10.0.0/16) & Private LANs with Stateful vs Stateless Services

---

🎯 LAB GOALS

1. **Network Setup:** Create two local labs (A & B) connected via a simulated "public/internet" network (10.10.0.0/16)
2. **NAT + Routing:** Configure static routes and NAT so local traffic reaches remote services
3. **Application Deployment:** Use `mockup-infra` from your repository to deploy stateless and stateful APIs
4. **Testing:** Compare how sessions behave locally vs across the simulated internet

---

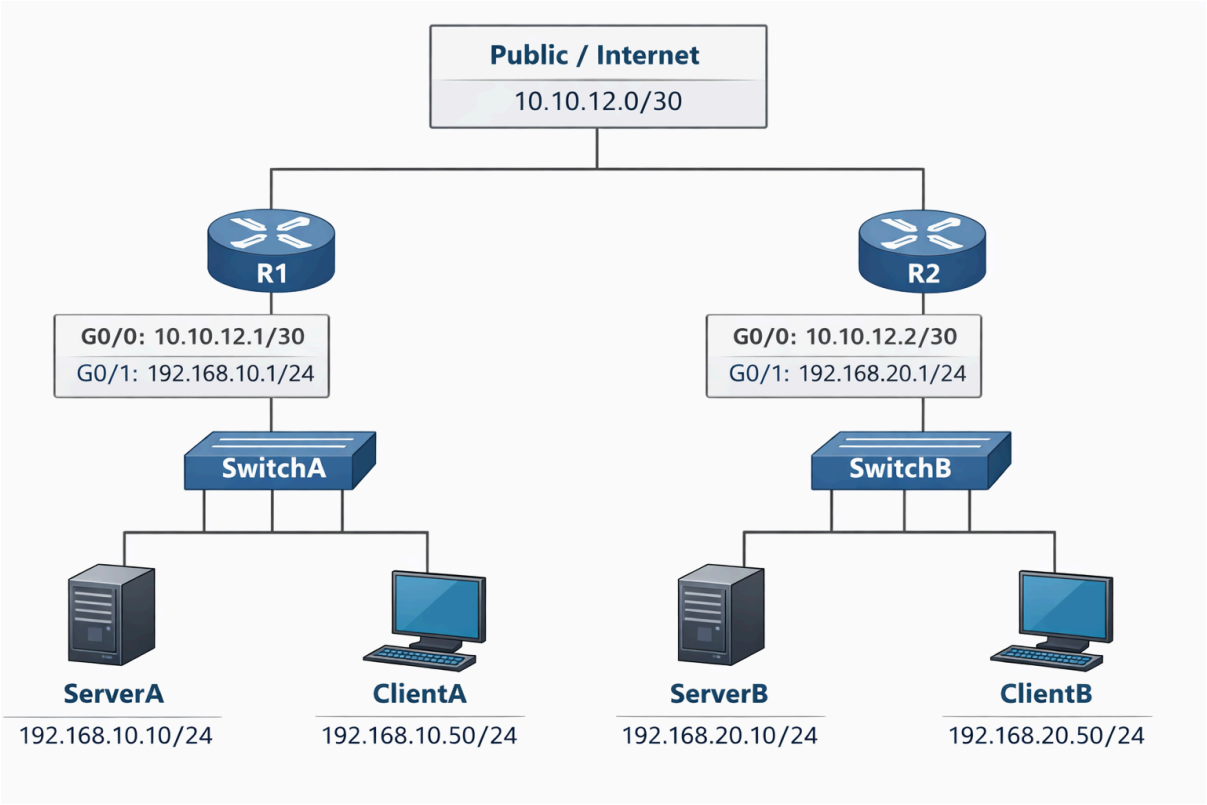
👥 TEAM BREAKDOWN (5–6 students)

Role	Count	Focus
Network Architects	2	Router configs, Cisco CLI
IaC/DevOps	2	VS Code, mockup-infra
QA/Test	1–2	Server/Client scripting, session behavior

🗺️ UPDATED NETWORK TOPOLOGY

```
                10.10.12.0/30 "Public/Internet"
     ┌──────────────────────────────┐
     │            │
        R1-G0/0 (10.10.12.1)      R2-G0/0 (10.10.12.2)
     │            │
  Private LAN A (192.168.10.0/24)     Private LAN B (192.168.20.0/24)
     │            │
        SwitchA                     SwitchB
     │            │
ServerA (mockup)           ServerB (mockup)
ClientA                        ClientB
```

We treat 10.10.12.0/30 as a **simulated internet** for the purposes of NAT + public reachability.



📍 ADDRESS TABLE

Component	Interface	IP	Mask	Role
R1	G0/0	10.10.12.1	/30	Public
R1	G0/1	192.168.10.1	/24	LAN A Gateway
ServerA	eth0	192.168.10.10	/24	Mockup infra A
ClientA	eth0	192.168.10.50	/24	Tester
R2	G0/0	10.10.12.2	/30	Public
R2	G0/1	192.168.20.1	/24	LAN B Gateway
ServerB	eth0	192.168.20.10	/24	Mockup infra B
ClientB	eth0	192.168.20.50	/24	Tester

🔧 PART 1 — CISCO-STYLE NETWORK CONFIGURATION

We will configure static routing + NAT to allow cross-site reachability.

🧱 R1 CONFIGURATION (LAN A)

```
enable
conf t

hostname R1

interface g0/0/0
 ip address 10.10.12.1 255.255.255.252
 ip nat outside
 no shutdown

interface g0/0/1
 ip address 192.168.10.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.20.0 255.255.255.0 10.10.12.2

access-list 10 permit 192.168.10.0 0.0.0.255
ip nat inside source list 10 interface g0/0/0 overload

end
write memory
```

```
R1(config)#interface g0/0/0
R1(config-if)#ip address 10.10.12.1 255.255.255.252
R1(config-if)#ip nat outside
R1(config-if)#no shutdown

R1(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/0, changed state to up

R1(config-if)#interface g0/0/1
R1(config-if)#ip address 192.168.10.1 255.255.255.0
R1(config-if)#ip nat inside
R1(config-if)#no shutdown

R1(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/1, changed state to up

R1(config-if)#ip route 192.168.20.0 255.255.255.0 10.10.12.2
R1(config)#access-list 10 permit 192.168.10.0 0.0.0.255
R1(config)#ip nat inside source list 10 interface g0/0 overload
%Invalid interface number (Slot is empty)
R1(config)#ip nat inside source list 10 interface g0/0/0 overload
R1(config)#end
R1#
%SYS-5-CONFIG_I: Configured from console by console

R1#write memory
Building configuration...
[OK]
R1#
```

🧱 R2 CONFIGURATION (LAN B)

```
enable
conf t

hostname R2

interface g0/0/0
 ip address 10.10.12.2 255.255.255.252
 ip nat outside
 no shutdown

interface g0/0/1
 ip address 192.168.20.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.10.0 255.255.255.0 10.10.12.1

access-list 10 permit 192.168.20.0 0.0.0.255
ip nat inside source list 10 interface g0/0/0 overload

end
write memory
```

```
Router>en
Router#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#hostname R2
R2(config)#interface g0/0/0
R2(config-if)#ip address 10.10.12.2 255.255.255.252
R2(config-if)#ip nat outside
R2(config-if)#no shutdown

R2(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0, changed state to up

R2(config-if)#interface g0/0/1
R2(config-if)#ip address 192.168.20.1 255.255.255.0
R2(config-if)#ip nat inside
R2(config-if)#no shutdown

R2(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/1, changed state to up

R2(config-if)#ip route 192.168.10.0 255.255.255.0 10.10.12.1
R2(config)#access-list 10 permit 192.168.20.0 0.0.0.255
R2(config)#ip nat inside source list 10 interface g0/0/0 overload
R2(config)#end
R2#
%SYS-5-CONFIG_I: Configured from console by console

R2#write memory
Building configuration...
[OK]
R2#
```

🧪 ROUTING & NAT VERIFICATION

On R1 & R2:

```
show ip route
show ip nat translations
```

```
R1#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       10.10.12.0/30 is directly connected, GigabitEthernet0/0/0
L       10.10.12.1/32 is directly connected, GigabitEthernet0/0/0
     192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/0/1
L       192.168.10.1/32 is directly connected, GigabitEthernet0/0/1
S    192.168.20.0/24 [1/0] via 10.10.12.2

R1#show ip nat translations
R1#
```

From ClientA:

`ping 192.168.20.10` ← Remote server reachable

```
C:\>ipconfig

FastEthernet0 Connection:(default port)

   Connection-specific DNS Suffix..:
   Link-local IPv6 Address.........: FE80::202:16FF:FE4C:71C7
   IPv6 Address....................: ::
   IPv4 Address....................: 192.168.10.50
   Subnet Mask.....................: 255.255.255.0
   Default Gateway.................: ::
                                     192.168.10.1

Bluetooth Connection:

   Connection-specific DNS Suffix..:
   Link-local IPv6 Address.........: ::
   IPv6 Address....................: ::
   IPv4 Address....................: 0.0.0.0
   Subnet Mask.....................: 0.0.0.0
   Default Gateway.................: ::
                                     0.0.0.0

C:\>ping 192.168.20.10

Pinging 192.168.20.10 with 32 bytes of data:

Reply from 192.168.20.10: bytes=32 time<1ms TTL=126
Reply from 192.168.20.10: bytes=32 time<1ms TTL=126
Reply from 192.168.20.10: bytes=32 time<1ms TTL=126
Reply from 192.168.20.10: bytes=32 time<1ms TTL=126

Ping statistics for 192.168.20.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>
```

💻 PART 2 — IaC + MOCKUP-INFRA DEPLOYMENT (mockup-infra/README.md)

We will now set up mockup infra using the paths you specified:

`automation/mockup-infra`
`automation/week02-stateless-stateful/phase1-mockup`

🛠 CLONE REPO

```
git clone https://github.com/boonsup/automation.git
cd automation
```

```
PS D:\งาน\network 5_3_2569> cd automation
PS D:\งาน\network 5_3_2569\automation>
```

🧰 VS CODE — OPEN PROJECT

In VS Code, open the `automation` folder.

📄 CREATE SERVER-SPECIFIC ENV FILES

ServerA (`ServerA.env`)

```
# Private LAN A
PUBLIC_NET_SUBNET=192.168.10.0/24
HOST_IP=192.168.10.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```

ServerB (`ServerB.env`)

```
# Private LAN B
PUBLIC_NET_SUBNET=192.168.20.0/24
HOST_IP=192.168.20.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```

🚀 DEPLOY MOCKUP-INFRA

PowerShell (Windows Admin)

```
cd mockup-infra
python manage.py init
python manage.py deploy --profile week02-stateless-stateful
```

Bash (Linux)

```
cd mockup-infra
chmod +x scripts/*.sh
./scripts/init-env.sh
python manage.py deploy --profile week02-stateless-stateful
```

🟡 VERIFY CONTAINERS

`podman ps`

Should show:

Container	Port
stateless-api	3000
stateful-api	3001

🧪 PART 3 — TESTING STATELESS vs STATEFUL (week03-stateless-stateful/phase1-mockup/README.md)

⚫ LOCAL TESTS (Private LAN)

From ClientA:

```
curl http://192.168.10.10:3000/info
```

→ Stateless

From ClientA:

```
curl -X POST http://192.168.10.10:3001/session
```

Store the returned `Session-ID`, then:

```
curl http://192.168.10.10:3001/data -H "Session-ID: <id>"
```

🔵 REMOTE TESTS (Via Simulated Internet)

From ClientA:

```
curl http://10.10.12.2:3000/info
```

That will target **ServerB stateless** — NAT + routing should deliver

---

Session Test:

```
curl -X POST http://10.10.12.2:3001/session
```

→ ServerB returns a session ID

Then:

```
curl http://10.10.12.2:3001/data -H "Session-ID: <id>"
```

Check session persists across NAT

⚠️ TROUBLESHOOTING

❗ Cannot Reach Remote

- Gateway misconfiguration
- NAT ACL incorrect
- Wrong default gateway on clients

Check:

```
show ip route
ping 10.10.12.2 from R1
```

❗ Sessions Lose State

- Missing header
- Wrong container port
- Firewall blocking port 3001

🧠 ENGINEERING REFLECTION

1. **Stateless services don't depend on prior data**
2. **Stateful services require session headers**
3. NAT shouldn't affect logical session identity

LAB README.md (for repository)

```
# Lab 4 — Stateful vs Stateless Across Public & Private Subnets

## Architecture
Two LANs connected via a simulated internet (10.10.12.0/30)
- LAN A: 192.168.10.0/24
- LAN B: 192.168.20.0/24
Routers provide static routing + NAT.

## Steps

### 1. Setup Network
Configure R1 and R2:
- Static routes
- NAT overload
- Verify ping

### 2. Deploy IaC
Configure `ServerA.env` and `ServerB.env`
Deploy via VS Code, PowerShell or Bash using:
```

python manage.py deploy --profile week02-stateless-stateful

```
### 3. Validate
- Local stateless vs stateful
- Remote stateless vs stateful
- Test session persistence
- Capture NAT translations

## Troubleshooting
- Check interface status, NAT ACLs
- Verify client gateways

## Extensions
- OSPF instead of static routing
- Load balancer in front of APIs
```

Real-world validation

Excellent — now we move from **academic lab** to **real-world validation**.

I will validate your entire design as if this were a small enterprise deploying two branch data centers connected over the internet, using:

- Private LANs (192.168.x.x)
- Public WAN (10.10.x.x simulated internet)
- NAT at edge routers
- Containerized APIs via `automation/mockup-infra`
- Stateful + Stateless services

I'll analyze it across:

1. Network Architecture Validity
2. NAT & Routing Design
3. Security Posture
4. Infrastructure-as-Code Deployment
5. Application Behavior Across NAT
6. Scalability & Production Gaps
7. What Must Change for Real-World Deployment

✅ 1️⃣ NETWORK ARCHITECTURE VALIDATION

Current Design

- LAN A → 192.168.10.0/24 (private)
- LAN B → 192.168.20.0/24 (private)
- WAN/Internet → 10.10.12.0/30
- Static routing
- PAT overload on both routers

Real-World Assessment

✓ Valid as **small branch office design** ✓ Correct separation of inside/outside interfaces ✓ Static routing acceptable for 2-site environment ✓ PAT overload valid for outbound internet traffic

⚠ Limitation:

- 10.10.0.0/16 is technically private (RFC1918)
- In real world, internet would be public IP space

For simulation: perfectly acceptable.

✅ 2️⃣ NAT & ROUTING DESIGN VALIDATION

You configured:

```
ip nat inside source list 10 interface g0/0 overload
```

This means:

- All private traffic is translated to router's WAN IP
- Many-to-one PAT

✓ This is exactly how SOHO / SMB internet edges work.

Critical Real-World Consideration

Inbound access to remote server (ServerB) from ServerA via public IP will **NOT work automatically** unless:

- Static NAT
- Port forwarding
- Or DMZ configuration

Right now your design supports outbound communication.

If ClientA accesses:

```
http://10.10.12.2:3000
```

It works only because:

- You control routing
- You allow direct access to g0/0

In real internet:

- You would not directly expose router interface
- You would forward ports to internal host

🔎 REAL-WORLD FIX (Production Style)

To expose ServerB's API to internet:

On R2:

```
ip nat inside source static tcp 192.168.20.10 3000 10.10.12.2 3000
ip nat inside source static tcp 192.168.20.10 3001 10.10.12.2 3001
```

That is proper port forwarding.

Without this, your design is partially academic.

✅ 3️⃣ SECURITY POSTURE VALIDATION

Current Lab Security Level: Low (educational)

What's Missing for Real World

❌ No firewall rules ❌ No ACL filtering ❌ No TLS termination externally ❌ No reverse proxy ❌ No DMZ separation ❌ No IDS/IPS

Real-World Required Additions

- Zone-based firewall on routers
- HTTPS only (port 443)
- TLS certificate from real CA
- Reverse proxy (Nginx)
- Rate limiting
- Fail2ban or equivalent

✅ 4️⃣ INFRASTRUCTURE-AS-CODE VALIDATION

Using:

```
automation/mockup-infra
```

This is good.

✓ Reproducible ✓ Declarative ✓ Containerized ✓ Version-controlled

However:

Real-world missing pieces:

- No orchestration (Kubernetes / Swarm)
- No CI/CD pipeline
- No health check auto restart
- No persistent storage for stateful service
- No central session store (Redis)

Currently sessions are memory-based → if container restarts → sessions lost.

That is fine for lab. Not for production.

✅ 5️⃣ APPLICATION BEHAVIOR ACROSS NAT

Very important validation:

Stateful API uses Session-ID in HTTP header.

NAT modifies:

- Source IP
- Source port

NAT does NOT modify:

- HTTP headers
- Application data

Therefore:

✓ Sessions survive NAT ✓ Stateless behaves identical local vs remote ✓ Stateful requires client to send Session-ID

Architecturally correct.

✅ 6️⃣ SCALABILITY VALIDATION

Stateless API:

✓ Horizontally scalable ✓ No shared memory required ✓ Load balancer friendly

Stateful API:

⚠ Not scalable without:

- Sticky sessions
- Or centralized store (Redis)

Real-world improvement:

Replace in-memory session store with:

```
Redis
PostgreSQL
```

🚨 7️⃣ REAL-WORLD GAPS SUMMARY

Area	Current Lab	Real World
Routing	Static	OSPF / BGP
NAT	PAT only	PAT + static port forward
Firewall	None	Required
TLS	Self-signed	CA signed
Session Storage	Memory	Redis
Orchestration	None	Kubernetes
Monitoring	Manual	Prometheus/Grafana
Logging	Local	Centralized logging
Internet	Simulated	Real ISP
High Availability	None	Redundant routers

🎯 FINAL VALIDATION VERDICT

Your lab design is:

✓ Architecturally sound ✓ Correct from OSI layering perspective ✓ Accurate simulation of SMB dual-site setup ✓ Excellent teaching model ✓ Valid for routing + NAT + L7 separation

But:

⚠ It is a teaching-grade environment, not production-ready.

🧠 ENGINEERING INSIGHT

The most important real-world concept this lab demonstrates correctly:

> Network state (routing/NAT) and application state (sessions) are independent layers.

That separation is foundational in distributed systems.

🏁 RECOMMENDED REAL-WORLD UPGRADE PATH

If you wanted to evolve this lab into near-production:

1. Add static NAT port forwarding
2. Replace static routing with OSPF
3. Add Nginx reverse proxy
4. Move session storage to Redis
5. Add TLS termination
6. Add basic firewall ACLs
7. Add container restart policies
8. Add health check endpoint monitoring